

EVENTHIVE

Event Booking System

Mini Project Proposal · Version 1.0 · August 2026

1. Overview

EventHive is an event booking platform that lets an attendee browse upcoming events and reserve tickets, all from a single web application backed by a REST API. Centralising ticket inventory and booking logic in one backend service keeps seat counts accurate and prevents overbooking.

2. System Components

The platform is composed of the following two pieces:

Component	Type	Responsibilities
EventHive Web App	Web application	Event browsing, ticket booking, and booking history — used directly by the attendee.
EventHive Core	Backend web service (REST API)	Central business logic, authentication, and data persistence — consumed by the web app.

3. User Role

This is a single-role application. The Attendee is the only user type, served entirely by EventHive Web App.

Attendee — served by EventHive Web App

- Create an account and log in.
- Browse and search upcoming events.
- Select a ticket type and book tickets for an event.
- View and cancel their own bookings.

4. Database Design

A relational schema of four tables backs the platform. A shared users table handles authentication, events and their ticket types are kept separate so an event can offer several price tiers, and a bookings table records every ticket purchase against the attendee who made it.

users

Column	Type	Description
id	BIGINT (PK)	Unique identifier, auto-generated.
email	VARCHAR(150)	Login email address; unique.
password_hash	VARCHAR(255)	Securely hashed password.
role	VARCHAR(20)	Fixed as ATTENDEE.

Column	Type	Description
created_at	TIMESTAMP	When the account was created.

events

Column	Type	Description
event_id	BIGINT (PK)	Unique event identifier.
title	VARCHAR(200)	Event title.
description	VARCHAR(500)	Event description.
venue	VARCHAR(200)	Location of the event.
event_date	TIMESTAMP	Date and time of the event.
category	VARCHAR(50)	Music, sports, conference, etc.

ticket_types

Column	Type	Description
ticket_type_id	BIGINT (PK)	Unique ticket-type identifier.
event_id	BIGINT (FK)	References events.event_id.
name	VARCHAR(50)	General, VIP, Early Bird, etc.
price	DECIMAL(10,2)	Price per ticket.
total_quantity	INT	Total tickets available for this type.
quantity_sold	INT	Tickets sold so far.

bookings

Column	Type	Description
booking_id	BIGINT (PK)	Unique booking identifier.
user_id	BIGINT (FK)	References users.id.
ticket_type_id	BIGINT (FK)	References ticket_types.ticket_type_id.
quantity	INT	Number of tickets booked.
total_amount	DECIMAL(10,2)	Total price paid.
status	VARCHAR(20)	CONFIRMED or CANCELLED.
booked_at	TIMESTAMP	When the booking was made.

5. API Endpoints

All endpoints are exposed by EventHive Core.

Method	Endpoint	Description
POST	/api/auth/register	Register a new attendee account.

Method	Endpoint	Description
POST	/api/auth/login	Authenticate the attendee and issue a token.
GET	/api/events	List and search upcoming events.
GET	/api/events/{eventId}	Get full details and ticket types for one event.
POST	/api/bookings	Book tickets for an event.
GET	/api/bookings/user/{userId}	List all bookings made by an attendee.
PUT	/api/bookings/{bookingId}/cancel	Cancel a booking.

6. UI Pages (excluding Login)

Besides the standard login screen, attendees use the following three pages:

Page	Description
Event Listings	Browse and filter upcoming events by category, date, or keyword.
Event Details & Booking	View event details, choose a ticket type and quantity, and confirm the booking.
My Bookings	List of the attendee's bookings with status, and an option to cancel.

7. Suggested Technology Stack

The choices below are a suggestion that fits the REST design above; they can be swapped for whatever stack the team prefers.

- EventHive Core: a REST backend (e.g. Java + Spring Boot or Node.js + Express) exposing the endpoints above.
- Database: a relational database such as MySQL or PostgreSQL for the tables above.
- EventHive Web App: a web front-end (e.g. React or Angular single-page app) that calls EventHive Core.
- Authentication: token-based (e.g. JWT) securing every endpoint behind login.

8. End-to-End Flow

1. A prospective attendee registers via POST /api/auth/register and logs in via POST /api/auth/login.
2. The attendee browses events (GET /api/events) and opens one to see ticket types (GET /api/events/{eventId}).
3. The attendee books tickets (POST /api/bookings); EventHive Core reduces the available quantity and sets status to CONFIRMED.
4. The attendee can return anytime to view their bookings (GET /api/bookings/user/{userId}).
5. If plans change, the attendee cancels a booking (PUT /api/bookings/{bookingId}/cancel), and EventHive Core restores the ticket quantity.